



Universidade Federal do Ceará

Professor: Sidartha Carvalho

Disciplina: Desenvolvimento de Software para Web

Curso: \_\_\_\_\_

Aluno: \_\_\_\_\_

Matrícula: \_\_\_\_\_

Turma: \_\_\_\_\_

Nota

Data: \_\_/\_\_/\_\_

Marque o gabarito preenchendo completamente a região de cada alternativa.



Prova:

- ☐ a ☐ b ☐ c ☐ d
- Q.1: Discursiva  
Q.2: Discursiva  
Q.3:  
Q.4:  
Q.5:  
Q.6:  
Q.7:  
Q.8:  
Q.9:  
Q.10:  
Q.11:  
Q.12:  
Q.13:  
Q.14:  
Q.15:  
Q.16:  
Q.17:  
Q.18:  
Q.19:  
Q.20:  
Q.21:  
Q.22:

a b c d

☐ ☐

**Q.1 (1.50)** - Crie o código JavaScript para listar os elementos que são retornados em um JSON que é adquirido ao realizar o download dos dados na seguinte URL: <http://www.test.com/personagens>. Dica: Use fetch e Map, imprima no console.log o nome e o KI de cada personagem.

Exemplo de JSON:

```
{
  "nome": "Goku",
  "KI": 8001,
  "url": "https://exemplo.com/imagens/goku.jpg"
```

```
},
{
  "nome": "Vegeta",
  "KI": 18000,
  "url": "https://exemplo.com/imagens/vegeta.jpg"
}]
```

**Q.2 (1.50)** - Dado um array de dicionários chamado produtos contendo as propriedades literais dinâmicas nome (String) e preco (Number), escreva um código usando a iteração nativa `map()` para criar uma constante nomeada como `produtosComAumento`.

Verifique as respostas em: [www.gradepen.com/?ansid=](http://www.gradepen.com/?ansid=)



Na função de callback (uma arrow implícita), utilize a regra do Spread Operator para preservar inteiramente todos os dados passados, mas sobrescreva estritamente a chave interna referente a preço aplicando 10% de aumento no valor.

**Q.3 (0.35)** - Qual trecho adiciona um método nativo chamado de novaFunc à objetos do tipo String legada?

- a) ☐ String.addPrototype("novaFunc", ()=> {})
- b) ☒ String.prototype.novaFunc = function() {}
- c) ☐ String.prototype = { novaFunc() {} }
- d) ☐ String.\_\_proto\_\_.novaFunc = function() {}

**Q.4 (0.35)** - Bibliotecas nativas estritas como os atuadores Math.max() rejeitam processamento linear via coleções inteiras como os Arrays, aceitando argumentos estritos singulares. A abordagem recomendada, funcional e expansiva capaz de explodir o bloco arr nela é:

- a) ☐ Math.max([arr])
- b) ☐ Math.max(arr[])
- c) ☐ Math.max(&arr)
- d) ☒ Math.max(...arr)

**Q.5 (0.35)** - Com as funcionalidades de desestruturação literal profunda em Javascript atual podemos puxar valores diretamente nomeando suas chaves mapeadas. Mas ao precisarmos renomear a coluna nome para primNome durante a coleta para evitar conflitos:

- a) ☐ A) const { primNome: nome } = obj
- b) ☒ B) const [ nome: primNome ] = obj
- c) ☐ C) const { nome } = obj.primNome
- d) ☐ D) const { nome: primNome } = obj

**Q.6 (0.35)** - Por recomendação imutável de boas práticas devemos evitar com vigor extremo a sintaxe:

- a) ☐ A) Manipulação da renderização em querySelector.
- b) ☒ B) Chamadas cruas dentro da instrução de eval()
- c) ☐ C) Processos em cadeia dentro da instrução fetch
- d) ☐ D) Compilação estrita pela instrução de execução.

**Q.7 (0.35)** - Os parâmetros e as entradas em JS são incrivelmente relaxados comparados a linguagens tipadas rigorosamente. Ao instanciar function add(x, y) ignorando operadores Rest e invocarmos descaramadamente na execução add(2, 3, 4) o que o interpretador devolve?

- a) ☒ A) O terceiro parâmetro é ignorado..
- b) ☐ B) A função soma todos os elementos.
- c) ☐ C) O resultado final se torna NaN...
- d) ☐ D) O script gera um erro de execução.

**Q.8 (0.35)** - Ao tentar acessar as propriedades CSS com o JavaScript, temos um problema com o uso dos hifens (-). Para contornar e ajustar:

- a) ☒ A) Seguir os nomes no formato padrão camel-Case.
- b) ☐ B) Que os hifens sejam estritamente deletados.
- c) ☐ C) Que a propriedade exija declaração em bytes.
- d) ☐ D) Que os hifens sejam convertidos para o \_.

**Q.9 (0.35)** - O Encadeamento Opcional (?.) e a Coalescência Nula (??) são atalhos para evitar falhas silenciosas. Qual expressão acessa o nó da rua sem gerar interrupção e retorna o padrão estrito "N/A" se a busca terminar vazia?

- a) ☐ A) user.endereco?.rua ?? "N/A"
- b) ☐ B) user!endereco!.rua || "N/A"
- c) ☒ C) user?.endereco?.rua ?? "N/A"
- d) ☐ D) user?.endereco?.rua || "N/A"

**Q.10 (0.35)** - Para tornar uma variável privada no contexto da classe no JS, usamos:

- a) ☐ A) O símbolo de sublinhado (\_).
- b) ☐ B) O símbolo de cifrão (\$).
- c) ☐ C) A palavra-chave private.
- d) ☒ D) O símbolo de cerquilha (#).

**Q.11 (0.35)** - O advento da ferramenta querySelector() absorveu sintaxes flexíveis vindas dos metadados globais via seletores de renderização do Webkit. Ao disparar uma consulta "info" recebemos:

- a) ☐ A) O último elemento que satisfaz à avaliação.
- b) ☒ B) O primeiro elemento que satisfaz a query.
- c) ☐ C) Um valor nulo informando alerta silencioso
- d) ☐ D) Um array embutido com todos os elementos...

**Q.12 (0.35)** - A flexibilidade extrema do operador solto de verificação binária no Javascript (==) oculta a auto conversão em background destruindo as referências puras entre String vs Number. Se quisermos uma aplicação bancária segura forçando o rigor de paridade e travar estritamente avaliações falsas perigosas, a comparação sintática imperativa da verificação de tipos deverá injetar e invocar o método comumente referenciado em:

- a) ☐ A) Conversões estáticas em bytes binários fortes.
- b) ☐ B) Chamadas lógicas da verificação assíncrona
- c) ☒ C) Acionamento sintático do Igual Estrito (===)
- d) ☐ D) Injeção de verificação via função (Object())

**Q.13 (0.35)** - O JavaScript provê diferentes laços de repetição que facilitam iterações na base das estruturas de interface de dados. Para percorrer unicamente os atributos (índices/chaves nominais) de um dado objeto dinâmico literal, use:

- a) ☐ A) for (let i = 0; obj)

Verifique as respostas em: [www.gradepen.com/?ansid=](http://www.gradepen.com/?ansid=)



- b) ☐ while(key from obj)
- c) ☐ for (let key of obj)
- d) ☒ for (let key in obj)

**Q.14 (0.35)** - Como a classe Cachorro faz para herdar todos os comportamentos de Animal?

- a) ☐ class Cachorro inherits Animal { }
- b) ☐ class Cachorro super Animal { }
- c) ☒ class Cachorro extends Animal { }
- d) ☐ class Cachorro implements Animal { }

**Q.15 (0.35)** - Como escrever um acessador e interceptador para gravar dados limpos no encapsulado embutido de #preco?

- a) ☒ set #preco(v) { if(v>0) this.preco = v; }
- b) ☐ setter preco(v) { if(v>0) #preco = v; }
- c) ☒ setPreco(v) { if(v>0) this.#preco = v; }
- d) ☐ set preco(v) { if(v>0) this.#preco = v; }

**Q.16 (0.35)** - O JavaScript avalia expressões da esquerda para a direita. Para evitar problemas de coerção e garantir que os números sejam somados matematicamente antes de uma string ser processada, qual a abordagem correta na sintaxe?

- a) ☒ let res = (16 + 4) + "Str";
- b) ☐ let res = 16 + (4 + "Str");
- c) ☐ let res = "Str" + (16 + 4);
- d) ☐ let res = Number(16+4+"S");

**Q.17 (0.35)** - O operador ternário atua como um bloco condicional compacto (condição ? verdadeiro : falso). Qual a sintaxe correta para retornar em string "yes" caso a proposição age > 18 seja verdadeira, e "no" caso contrário?

- a) ☒ age > 18 ? "yes" : "no";
- b) ☐ age > 18 ? "no" : "yes";
- c) ☐ age > 18 : "yes" ? "no";
- d) ☐ age > 18 = "yes" | "no";

**Q.18 (0.35)** - Diversos métodos funcionais de manipulação podem modificar o escopo, ordem ou os índices associados em um array genérico. Qual exato par encadeia a remoção imperativa do último item e adiciona um elemento no novo final da fila?

- a) ☐ push() e shift()
- b) ☒ pop() e push()
- c) ☐ slice() e push()
- d) ☐ shift() e pop()

**Q.19 (0.35)** - Qual declarador aplica a pausa de barreira aguardando o carregamento da requisição nativa sem que trave renderizadores em processo paralelo na thread gráfica original do frontend?

- a) ☐ yield
- b) ☒ await
- c) ☐ B) pause
- d) ☐ defer

**Q.20 (0.35)** - Qual alternativa é a forma mais indicada de alterar o endereço de uma imagem dinamicamente?

- a) ☐ document.insertAttribute('src');
- b) ☒ document.setAttribute('src', 'xxx');
- c) ☐ document.setProp('src', 'xxx');
- d) ☐ document.getAttribute('src', 'xxx');

**Q.21 (0.35)** - Caso repassarmos n valores flutuantes dinâmicos e contínuos para uma atribuição cega encapsulada em var avg = function(), como inspecionaremos dentro da declaração base quantos índices estritos constam contidos lá sem ter a referência formal:

- a) ☐ variables.length
- b) ☐ functions.length
- c) ☒ arguments.length
- d) ☐ parameter.length

**Q.22 (0.35)** - A notação de texto JSON (JavaScript Object Notation) é muito utilizada para troca leve de tráfego de dados remotos. Qual das strings estruturadas representa corretamente um array que encapsula um objeto JSON válido?

- a) ☐ { ( "nome": "Ana" ) }
- b) ☒ [ { "nome": "Ana" } ]
- c) ☐ { [ "nome": "Ana" ] }
- d) ☐ [ { 'nome': 'Ana' } ]